All this was hard-wired into the code.

We kept an index on the disk that allowed us to look up each of the `Agents`, `Employers`, and `Members`. This index was in yet another specially formatted set of cylinders on the disk. The `Agent` index was composed of records that contained the ID of an agent, and the cylinder number, head number, and sector number of that `Agent` record. `Employers` and `Members` had similar indices. `Members` were also kept in a doubly linked list on the disk. Each `Member` record held the cylinder, head, and sector number of the next `Member` record, and of the previous `Member` record.

What would happen if we needed to upgrade to a new disk drive—one with more heads, or one with more cylinders, or one with more sectors per cylinder? We had to write a special program to read in the old data from the old disk, and then write it out to the new disk, translating all of the cylinder/head/sector numbers. We also had to change all the hard-wiring in our code—and that hard-wiring was *everywhere*! All the business rules knew the cylinder/head/sector scheme in detail.

One day a more experienced programmer joined our ranks. When he saw what we had done, the blood drained from his face, and he stared aghast at us, as if we were aliens of some kind. Then he gently advised us to change our addressing scheme to use relative addresses.

Our wiser colleague suggested that we consider the disk to be one huge linear array of sectors, each addressable by a sequential integer. Then we could write a little conversion routine that knew the physical structure of the disk, and could translate the relative address to a cylinder/head/sector number on the fly.

Fortunately for us, we took his advice. We changed the high-level policy of the system to be agnostic about the physical structure of the disk. That allowed us to decouple the decision about disk drive structure from the application.

CONCLUSION

The two stories in this chapter are examples, in the small, of a principle that architects employ in the large. Good architects carefully separate details from policy, and then decouple the policy from the details so thoroughly that the policy has no knowledge of the details and does not depend on the details in any way. Good architects design the policy so that decisions about the details can be delayed and deferred for as long as possible.

16
INDEPENDENCE

As we previously stated, a good architecture must support:

- The use cases and operation of the system.
- The maintenance of the system.
- The development of the system.
- The deployment of the system.

USE CASES

The first bullet—use cases—means that the architecture of the system must support the intent of the system. If the system is a shopping cart application, then the

architecture must support shopping cart use cases. Indeed, this is the first concern of the architect, and the first priority of the architecture. The architecture must support the use cases.

However, as we discussed previously, architecture does not wield much influence over the behavior of the system. There are very few behavioral options that the architecture can leave open. But influence isn't everything. The most important thing a good architecture can do to support behavior is to clarify and expose that behavior so that the intent of the system is visible at the architectural level.

A shopping cart application with a good architecture will *look* like a shopping cart application. The use cases of that system will be plainly visible within the structure of that system. Developers will not have to hunt for behaviors, because those behaviors will be first-class elements visible at the top level of the system. Those elements will be classes or functions or modules that have prominent positions within the architecture, and they will have names that clearly describe their function.

Chapter 21, "Screaming Architecture," will make this point much clearer.

OPERATION

Architecture plays a more substantial, and less cosmetic, role in supporting the operation of the system. If the system must handle 100,000 customers per second, the architecture must support that kind of throughput and response time for each use case that demands it. If the system must query big data cubes in milliseconds, then the architecture must be structured to allow this kind of operation.

For some systems, this will mean arranging the processing elements of the system into an array of little services can be run in parallel on many different servers. For other systems, it will mean a plethora of little lightweight threads sharing the address space of a single process within a single processor. Still other systems will need just a few processes running in isolated address spaces. And some systems can even survive as simple monolithic programs running in a single process.

As strange as it may seem, this decision is one of the options that a good architect leaves open. A system that is written as a monolith, and that depends on that monolithic structure, cannot easily be upgraded to multiple processes, multiple threads, or micro-services should the need arise. By comparison, an architecture that maintains the proper isolation of its components, and does not assume the means of

communication between those components, will be much easier to transition through the spectrum of threads, processes, and services as the operational needs of the system change over time.

DEVELOPMENT

Architecture plays a significant role in supporting the development environment. This is where Conway's law comes into play. Conway's law says:

> *Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure.*

A system that must be developed by an organization with many teams and many concerns must have an architecture that facilitates independent actions by those teams, so that the teams do not interfere with each other during development. This is accomplished by properly partitioning the system into well-isolated, independently developable components. Those components can then be allocated to teams that can work independently of each other.

DEPLOYMENT

The architecture also plays a huge role in determining the ease with which the system is deployed. The goal is "immediate deployment." A good architecture does not rely on dozens of little configuration scripts and property file tweaks. It does not require manual creation of directories or files that must be arranged just so. A good architecture helps the system to be immediately deployable after build.

Again, this is achieved through the proper partitioning and isolation of the components of the system, including those master components that tie the whole system together and ensure that each component is properly started, integrated, and supervised.

LEAVING OPTIONS OPEN

A good architecture balances all of these concerns with a component structure that mutually satisfies them all. Sounds easy, right? Well, it's easy for me to write that.

The reality is that achieving this balance is pretty hard. The problem is that most of the time we don't know what all the use cases are, nor do we know the operational constraints, the team structure, or the deployment requirements. Worse, even if we did know them, they will inevitably change as the system moves through its life cycle. In short, the goals we must meet are indistinct and inconstant. Welcome to the real world.

But all is not lost: Some principles of architecture are relatively inexpensive to implement and can help balance those concerns, even when you don't have a clear picture of the targets you have to hit. Those principles help us partition our systems into well-isolated components that allow us to leave as many options open as possible, for as long as possible.

A good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.

DECOUPLING LAYERS

Consider the use cases. The architect wants the structure of the system to support all the necessary use cases, but does not know what all those use cases are. However, the architect *does* know the basic intent of the system. It's a shopping cart system, or it's a bill of materials system, or it's an order processing system. So the architect can employ the Single Responsibility Principle and the Common Closure Principle to separate those things that change for different reasons, and to collect those things that change for the same reasons—given the context of the intent of the system.

What changes for different reasons? There are some obvious things. User interfaces change for reasons that have nothing to do with business rules. Use cases have elements of both. Clearly, then, a good architect will want to separate the UI portions of a use case from the business rule portions in such a way that they can be changed independently of each other, while keeping those use cases visible and clear.

Business rules themselves may be closely tied to the application, or they may be more general. For example, the validation of input fields is a business rule that is closely tied to the application itself. In contrast, the calculation of interest on an account and the counting of inventory are business rules that are more closely associated with the domain. These two different kinds of rules will change at different rates, and for different reasons—so they should be separated so that they can be independently changed.

The database, the query language, and even the schema are technical details that have nothing to do with the business rules or the UI. They will change at rates, and for reasons, that are independent of other aspects of the system. Consequently, the architecture should separate them from the rest of the system so that they can be independently changed.

Thus we find the system divided into decoupled horizontal layers—the UI, application-specific business rules, application-independent business rules, and the database, just to mention a few.

DECOUPLING USE CASES

What else changes for different reasons? The use cases themselves! The use case for adding an order to an order entry system almost certainly will change at a different rate, and for different reasons, than the use case that deletes an order from the system. Use cases are a very natural way to divide the system.

At the same time, use cases are narrow vertical slices that cut through the horizontal layers of the system. Each use case uses some UI, some application-specific business rules, some application-independent business rules, and some database functionality. Thus, as we are dividing the system in to horizontal layers, we are also dividing the system into thin vertical use cases that cut through those layers.

To achieve this decoupling, we separate the UI of the add-order use case from the UI of the delete-order use case. We do the same with the business rules, and with the database. We keep the use cases separate down the vertical height of the system.

You can see the pattern here. If you decouple the elements of the system that change for different reasons, then you can continue to add new use cases without interfering with old ones. If you also group the UI and database in support of those use cases, so that each use case uses a different aspect of the UI and database, then adding new use cases will be unlikely to affect older ones.

DECOUPLING MODE

Now think of what all that decoupling means for the second bullet: operations. If the different aspects of the use cases are separated, then those that must run at a high throughput are likely already separated from those that must run at a low throughput.

If the UI and the database have been separated from the business rules, then they can run in different servers. Those that require higher bandwidth can be replicated in many servers.

In short, the decoupling that we did for the sake of the use cases also helps with operations. However, to take advantage of the operational benefit, the decoupling must have the appropriate mode. To run in separate servers, the separated components cannot depend on being together in the same address space of a processor. They must be independent services, which communicate over a network of some kind.

Many architects call such components “services” or “micro-services,” depending upon some vague notion of line count. Indeed, an architecture based on services is often called a service-oriented architecture.

If that nomenclature set off some alarm bells in your mind, don’t worry. I’m not going to tell you that SoA is the best possible architecture, or that micro-services are the wave of the future. The point being made here is that sometimes we have to separate our components all the way to the service level.

Remember, a good architecture leaves options open. *The decoupling mode is one of those options.*

Before we explore that topic further, let’s look to the other two bullets.

INDEPENDENT DEVELOP-ABILITY

The third bullet was development. Clearly when components are strongly decoupled, the interference between teams is mitigated. If the business rules don’t know about the UI, then a team that focuses on the UI cannot much affect a team that focuses on the business rules. If the use cases themselves are decoupled from one another, then a team that focuses on the `addOrder` use case is not likely to interfere with a team that focuses on the `deleteOrder` use case.

So long as the layers and use cases are decoupled, the architecture of the system will support the organization of the teams, irrespective of whether they are organized as feature teams, component teams, layer teams, or some other variation.

INDEPENDENT DEPLOYABILITY

The decoupling of the use cases and layers also affords a high degree of flexibility in deployment. Indeed, if the decoupling is done well, then it should be possible to hot-swap layers and use cases in running systems. Adding a new use case could be a simple as adding a few new jar files or services to the system while leaving the rest alone.

DUPLICATION

Architects often fall into a trap—a trap that hinges on their fear of duplication.

Duplication is generally a bad thing in software. We don't like duplicated code. When code is truly duplicated, we are honor-bound as professionals to reduce and eliminate it.

But there are different kinds of duplication. There is true duplication, in which every change to one instance necessitates the same change to every duplicate of that instance. Then there is false or accidental duplication. If two apparently duplicated sections of code evolve along different paths—if they change at different rates, and for different reasons—*then they are not true duplicates.* Return to them in a few years, and you'll find that they are very different from each other.

Now imagine two use cases that have very similar screen structures. The architects will likely be strongly tempted to share the code for that structure. But should they? Is that true duplication? Or it is accidental?

Most likely it is accidental. As time goes by, the odds are that those two screens will diverge and eventually look very different. For this reason, care must be taken to avoid unifying them. Otherwise, separating them later will be a challenge.

When you are vertically separating use cases from one another, you will run into this issue, and your temptation will be to couple the use cases because they have similar screen structures, or similar algorithms, or similar database queries and/or schemas. Be careful. Resist the temptation to commit the sin of knee-jerk elimination of duplication. Make sure the duplication is real.

By the same token, when you are separating layers horizontally, you might notice that the data structure of a particular database record is very similar to the data

structure of a particular screen view. You may be tempted to simply pass the database record up to the UI, rather than to create a view model that looks the same and copy the elements across. Be careful: This duplication is almost certainly accidental. Creating the separate view model is not a lot of effort, and it will help you keep the layers properly decoupled.

DECOUPLING MODES (AGAIN)

Back to modes. There are many ways to decouple layers and use cases. They can be decoupled at the source code level, at the binary code (deployment) level, and at the execution unit (service) level.

- **Source level.** We can control the dependencies between source code modules so that changes to one module do not force changes or recompilation of others (e.g., Ruby Gems).

  In this decoupling mode the components all execute in the same address space, and communicate with each other using simple function calls. There is a single executable loaded into computer memory. People often call this a monolithic structure.
- **Deployment level.** We can control the dependencies between deployable units such as jar files, DLLs, or shared libraries, so that changes to the source code in one module do not force others to be rebuilt and redeployed.

  Many of the components may still live in the same address space, and communicate through function calls. Other components may live in other processes in the same processor, and communicate through interprocess communications, sockets, or shared memory. The important thing here is that the decoupled components are partitioned into independently deployable units such as jar files, Gem files, or DLLs.
- **Service level**. We can reduce the dependencies down to the level of data structures, and communicate solely through network packets such that every execution unit is entirely independent of source and binary changes to others (e.g., services or micro-services).

What is the best mode to use?

The answer is that it’s hard to know which mode is best during the early phases of a project. Indeed, as the project matures, the optimal mode may change.

For example, it's not difficult to imagine that a system that runs comfortably on one server right now might grow to the point where some of its components ought to run on separate servers. While the system runs on a single server, the source-level decoupling might be sufficient. Later, however, it might require decoupling into deployable units, or even services.

One solution (which seems to be popular at the moment) is to simply decouple at the service level by default. A problem with this approach is that it is expensive and encourages coarse-grained decoupling. No matter how "micro" the micro-services get, the decoupling is not likely to be fine-grained enough.

Another problem with service-level decoupling is that it is expensive, both in development time and in system resources. Dealing with service boundaries where none are needed is a waste of effort, memory, and cycles. And, yes, I know that the last two are cheap—but the first is not.

My preference is to push the decoupling to the point where a service *could* be formed. should it become necessary; but then to leave the components in the same address space as long as possible. This leaves the option for a service open.

With this approach, initially the components are separated at the source code level. That may be good enough for the duration of the project's lifetime. If, however, deployment or development issues arise, driving some of the decoupling to a deployment level may be sufficient—at least for a while.

As the development, deployment, and operational issues increase, I carefully choose which deployable units to turn into services, and gradually shift the system in that direction.

Over time, the operational needs of the system may decline. What once required decoupling at the service level may now require only deployment-level or even source-level decoupling.

A good architecture will allow a system to be born as a monolith, deployed in a single file, but then to grow into a set of independently deployable units, and then all the way to independent services and/or micro-services. Later, as things change, it should allow for reversing that progression and sliding all the way back down into a monolith.

A good architecture protects the majority of the source code from those changes. It leaves the decoupling mode open as an option so that large deployments can use one

mode, whereas small deployments can use another.

CONCLUSION

Yes, this is tricky. And I'm not saying that the change of decoupling modes should be a trivial configuration option (though sometimes that *is* appropriate). What I'm saying is that the decoupling mode of a system is one of those things that is likely to change with time, and a good architect foresees and *appropriately* facilitates those changes.

17
BOUNDARIES: DRAWING LINES

Software architecture is the art of drawing lines that I call *boundaries*. Those boundaries separate software elements from one another, and restrict those on one side from knowing about those on the other. Some of those lines are drawn very early in a project's life—even before any code is written. Others are drawn much later. Those that are drawn early are drawn for the purposes of deferring decisions for as long as possible, and of keeping those decisions from polluting the core business logic.

Recall that the goal of an architect is to minimize the human resources required to build and maintain the required system. What it is that saps this kind of people-power? *Coupling*—and especially coupling to premature decisions.

Which kinds of decisions are premature? Decisions that have nothing to do with the